

UN DIPLOMA **NFT** CON SU METADATA EN IPFS

Un ERC-721 con OpenZeppelin, una imagen y un JSON en IPFS, un despliegue verificado en Sepolia y los tres saltos —cadena, JSON, imagen— comprobados uno por uno.

MODALIDAD	DURACIÓN	REQUISITOS	META
En parejas	75 min en clase + lo que falte en casa	Node 22, repo del curso, MetaMask con Sepolia	11 pruebas verdes + 3 saltos resueltos

CÓMO LEER ESTA GUÍA

Está escrita para que la puedan seguir **sin ayuda**. Cada paso dice qué escribir, qué deberían ver y cómo saber que funcionó. Los comandos están en PowerShell (Windows, la sala); cuando cambian en macOS/Linux, se indica. Si algo no sale como aquí se describe, vayan a **«Errores frecuentes»** (sección 12) antes de pedir ayuda. Todo lo que se cita aquí se verificó al preparar el laboratorio, en septiembre de 2026.

REGLAS QUE NO ADMITEN EXCEPCIÓN

- Solo red local o Sepolia. **Nunca** dinero real.
- La frase semilla no se comparte con nadie, tampoco con el docente. No se firma lo que no se entiende.
- La clave privada y la URL del RPC van **solo** en el keystore de Hardhat (`npx hardhat keystore set ...`). Nunca en un archivo del repositorio.
- Ningún dato personal** en la cadena ni en IPFS (Ley 1581 de 2012): nada de nombres, cédulas, correos ni fotos de personas. Lo que se sube a IPFS es público y no se puede borrar.
- Los contratos son material de clase, sin auditar.

01 Objetivo, resultados de aprendizaje y entrega

LEER PRIMERO

Objetivo: implementar un token no fungible (ERC-721) con metadata descentralizada, y comprobar con evidencia propia que «el NFT» (una fila en un contrato) y «el archivo» (lo que vive en IPFS) son cosas distintas.

RA	QUÉ DEMUESTRA ESTE LABORATORIO
RA3	Implementar un estándar (ERC-721 con OpenZeppelin v5), con pruebas automatizadas y control de acceso y de suministro.
RA6	Integrar almacenamiento descentralizado: subir a IPFS, obtener CIDs, enlazarlos desde el contrato y resolverlos.

Qué se entrega (detalle en la lista de chequeo, sección 14):

EVIDENCIA	DETALLE	PESO
Pruebas	Captura de <code>11 passing de test/s11/DiplomaUSB.test.js</code> con SU contrato.	20 %
CIDs en IPFS	CID de la imagen y del JSON; ambos abren en una puerta pública.	20 %
Contrato en Sepolia	Dirección del contrato verificado y hash de la transacción de <code>emitir</code> .	25 %
Los tres saltos	Salida completa de <code>leer-diploma.js</code> y captura del token en un explorador.	20 %
Reflexión	Respuestas a las preguntas de la sección 13.	15 %

Política del curso: **sin evidencia en cadena (dirección o hash), el laboratorio no se califica.**

02 El mapa: qué van a hacer y por qué en este orden

PASO 3-4	contrato + pruebas	(sin internet)	15 min
PASO 5	imagen a IPFS	(Pinata)	10 min
PASO 6	JSON a IPFS	(Pinata)	10 min
PASO 7	ensayo en red local	(sin gastar nada)	10 min
PASO 8	despliegue en Sepolia	(ETH de prueba)	15 min
PASO 9-10	ver el token y los 3 saltos		15 min
PASO 11	reto opcional: el diploma como SBT		en casa

Lo que construyen, en una imagen:

```
EN LA CADENA (Sepolia)      EN IPFS      EN IPFS
ownerOf(1) = su dirección  -->  diploma.json  -->  diploma.png
tokenURI(1) = ipfs://CID_JSON  "image": "ipfs://CID_IMG"
```

El contrato guarda al dueño y **un texto**. Ese texto lleva al JSON; el JSON lleva a la imagen. Un explorador que «muestra el NFT» hace esos tres saltos por ustedes. En el paso 10 los harán ustedes a la vista.

SOBRE «VERLO EN UN MARKETPLACE DE PRUEBA»

El plan del curso pedía ver la colección en un marketplace de testnet. Lo verificamos: **OpenSea cerró su sitio para redes de prueba en julio de 2025** (`testnets.opensea.io` hoy redirige a un aviso de despedida). Por eso la evidencia es la vista del token en un **explorador** (Etherscan o Blockscout) más la metadata resuelta desde una **puerta IPFS** con el script del paso 10. Rarible mantiene un sitio de testnet (`testnet.rarible.com`) que lista colecciones de Sepolia; mirarlo es opcional y no depende de ustedes que indexe su contrato.

03 Preparación desde cero

ANTES DE LA CLASE

Abran una terminal (Windows: tecla Windows, escriban `PowerShell`, Enter) y comprueben:

```
node -v          # debe decir v22.algo
git --version    # cualquier versión reciente
```

Entren a la carpeta del repositorio del curso y comprueben Hardhat. Si nunca instalaron las dependencias, `npm install` primero (una sola vez en el semestre):

```
cd "ruta\a\laboratorios-evm"
npm install          # solo si no existe la carpeta node_modules
npx hardhat --version  # debe decir 3.16.0
```

Para Sepolia (pasos 8 a 10) necesitan lo mismo que en el laboratorio 8. Comprueben que las tres claves están en el keystore; si falta alguna, cárguenla (el comando la pide de forma oculta, no la escriban en el comando):

```
npx hardhat keystore list          # deben aparecer las tres
npx hardhat keystore set SEPOLIA_RPC_URL
npx hardhat keystore set SEPOLIA_PRIVATE_KEY
npx hardhat keystore set ETHERSCAN_API_KEY
```

Y ETH de Sepolia en la cuenta de la billetera del curso: con **0,02 ETH** sobra para este laboratorio (el despliegue midió 1 349 852 de gas y cada emisión unos 174 000). Si no tienen, pídanlo a la reserva del docente.

CÓMO SÉ QUE ESTOY LISTO

`npx hardhat --version` dice `3.16.0`, `keystore list` muestra las tres claves y MetaMask, en Sepolia, tiene saldo. Si están en la sala y algo de esto falla, pueden hacer igual los pasos 3 a 7 (no necesitan Sepolia) y el resto en casa.

Los archivos del laboratorio:

```
laboratorios-evm/
├── contracts/s11/DiplomaUSB.sol    <- AQUÍ TRABAJAN
├── andamiaje/s11/DiplomaUSB.sol    <- copia intacta con los TODO
├── andamiaje/s11/DiplomaSBT.sol    <- reto opcional (sección 11)
├── test/s11/DiplomaUSB.test.js     <- 11 pruebas. NO se modifica
├── ignition/modules/s11-DiplomaUSB.js <- despliega y emite
├── scripts/s11/leer-diploma.js     <- los tres saltos
└── scripts/s11/cid-raw.js          <- calcular un CID a mano
```

Paso 1. Abran `contracts/s11/DiplomaUSB.sol`. Si ya trae comentarios `// TODO`, trabajen ahí. Si no (o si lo dañaron y quieren empezar de nuevo), copien el andamiaje encima:

```
copy andamiaje\s11\DiplomaUSB.sol contracts\s11\DiplomaUSB.sol    # Windows
cp andamiaje/s11/DiplomaUSB.sol contracts/s11/DiplomaUSB.sol     # macOS / Linux
```

Paso 2. Corran las pruebas *antes* de escribir nada, para ver el punto de partida:

```
npx hardhat test test/s11/DiplomaUSB.test.js
```

Salida esperada (resumida): compila, y **4 pasan y 7 fallan**. Las que fallan son las marcadas con ★: dependen de lo que ustedes escriben.

```
1) ★ cada token tiene un id único y consecutivo
...
✓ el tope queda fijado al desplegar
5) ★ no se puede emitir por encima del tope
4 passing
7 failing
```

05 Paso 3 · Completar los TODO

4 TODO

1

TODO 1 · emitir() · CHECKS

Rechazar lo que no se puede emitir

Qué hace: antes de tocar el estado, revisa dos condiciones, en este orden: que el `uri` no esté vacío y que no se haya llegado al tope.

Qué escribir: `if (bytes(uri).length == 0) revert URIVacio();` y `if (_siguienteId > tope) revert TopeAlcanzado(tope);`

Por qué > y no >= : los ids empiezan en 1. Con tope 3, se pueden emitir los ids 1, 2 y 3; el cuarto intento tiene `_siguienteId = 4`, que es mayor que 3.

Cómo lo compruebo: pasan «no se puede emitir por encima del tope» y «rechaza un tokenURI vacío».

2

TODO 2 · emitir() · EFFECTS

Asignar el id y guardar el enlace

Qué escribir: `tokenId = _siguienteId++;` (usa el valor actual y luego lo incrementa) y `_setTokenURI(tokenId, uri);`

El detalle: `tokenId` es la variable de retorno con nombre: al asignarla, la función ya devuelve ese valor.

3

TODO 3 · emitir() · INTERACTION

Crear el token y avisar

Qué escribir: `_safeMint(a, tokenId);` y luego `emit DiplomaEmitido(tokenId, a, uri);`

Por qué al final: si `a` es un contrato, `_safeMint` le llama `onERC721Received`. Para entonces todo el estado ya debe estar escrito: es el patrón checks-effects-interactions de la Sesión 9.

Cómo lo compruebo: pasan las pruebas de ids, de tokenURI, de los dos eventos (`DiplomaEmitido` y el `Transfer` estándar desde la dirección cero) y la de transferencia.

4

TODO 4 · totalEmitidos()

Cuántos van

Qué escribir: `return _siguienteId - 1;` — si el siguiente id es 3, ya se emitieron el 1 y el 2.

Vuelvan a correr las pruebas. **Punto de control 1:**

```
npx hardhat test test/s11/DiplomaUSB.test.js
...
11 passing
```

Tomen la captura de esta salida: es la primera evidencia.

06 Paso 4 · Subir la imagen a IPFS

10 MIN · INTERNET

Qué servicio usar. Revisado en septiembre de 2026 en sus páginas de precios:

SERVICIO	PLAN GRATUITO ANUNCIADO	USO EN ESTE LABORATORIO
Pinata	1 GB, 500 archivos, una puerta propia (... mypinata.cloud).	Opción principal: <code>pinata.cloud</code> .
Filebase	5 GB compartidos, 500 archivos fijados en IPFS.	Alternativa: <code>filebase.com</code> .
IPFS Desktop	Su propio nodo, gratis, sin cuenta.	Plan B: el archivo solo está disponible mientras su PC esté encendido y conectado.

SI UN SERVICIO PIDE TARJETA DE CRÉDITO

No la pongan. El plan gratuito debe bastar; si el registro exige tarjeta, usen la alternativa o el plan B y avisen al docente. Registrarse pide un correo: usen uno que no les importe exponer a un tercero.

4.1 · Preparen la imagen. Cualquier PNG pequeño (menos de 1 MB) que NO tenga fotos ni datos de personas. Lo más simple: en Paint, un rectángulo con el texto `DIPLOMA DE PRUEBA · LAB 11 · USB`, guardado como `diploma.png`.

4.2 · Súbanla. En Pinata: *Files* → *Upload* → *File*, elijan el archivo. Cuando pregunte la red, elijan **Public** (público).

LA TRAMPA NÚMERO UNO: «PRIVATE»

Pinata permite subir a «Private IPFS». Según su propia documentación, esos archivos **no se anuncian a la red IPFS** y solo se abren con un enlace temporal: ninguna puerta pública ni ningún explorador los va a encontrar. Si su CID «no existe» en ninguna puerta, casi seguro lo subieron como privado. Súbanlo otra vez como público.

4.3 · Copien el CID. Pinata lo muestra junto al archivo. Los CID de versión 1 empiezan por `baf...` (por ejemplo `bafybei...` o `bafkrei...`); los antiguos, de versión 0, por `Qm...`. Péguenlo en un archivo de notas: lo necesitan en el paso siguiente.

4.4 · Ábranlo en una puerta. En el navegador:

```
https://gateway.pinata.cloud/ipfs/SU_CID_DE_LA_IMAGEN
```

Cómo sé que funcionó: el navegador muestra su imagen. Prueben también `https://ipfs.io/ipfs/SU_CID`: puede tardar la primera vez o responder `429 Too Many Requests` (límite de consultas de las puertas públicas; nos pasó al preparar el laboratorio). Un 429 no significa que el archivo no exista.

07 Paso 5 · Escribir y subir el JSON de metadata

10 MIN

5.1 · Creen el archivo `diploma.json` (en VS Code: *Archivo* → *Nuevo*, guardar con ese nombre). Pongan el CID de la imagen donde dice `SU_CID_DE_LA_IMAGEN`:

```
{
  "name": "Diploma de prueba · Laboratorio 11",
  "description": "Credencial de PRUEBA emitida en clase. No contiene datos personales.",
  "image": "ipfs://SU_CID_DE_LA_IMAGEN",
  "attributes": [
    { "trait_type": "Programa", "value": "Ingenieria de Sistemas" },
    { "trait_type": "Cohorte", "value": "2026-2" },
    { "trait_type": "Tipo", "value": "Prueba de laboratorio" }
  ]
}
```

CAMPO	DE DÓNDE SALE Y QUIÉN LO LEE
<code>name</code> , <code>description</code> , <code>image</code>	Esquema de metadata que propone el propio EIP-721. Lo leen exploradores y billeteras.
<code>attributes</code>	No es del estándar: es una convención que popularizó OpenSea y que hoy leen casi todos (los «rasgos»).
<code>"ipfs://..."</code>	Se escribe con el esquema <code>ipfs://</code> , no con la URL de una puerta: así no depende de ningún servidor.

5.2 · Comprueben que el JSON es válido antes de subirlo (un JSON roto es el error más tonto y más común):

```
node -e "console.log(JSON.parse(require('fs').readFileSync('diploma.json','utf8')).image)"
```

Debe imprimir `ipfs://SU_CID_DE_LA_IMAGEN`. Si dice `SyntaxError`, falta o sobra una coma o unas comillas.

5.3 · Súbanlo igual que la imagen (público). Copien su CID: **el tokenURI del diploma es**

`ipfs://SU_CID_DEL_JSON`.

5.4 · Punto de control 2: `https://gateway.pinata.cloud/ipfs/SU_CID_DEL_JSON` muestra el JSON, y copiando el CID que aparece en `"image"` en otra puerta, se ve la imagen.

EL ORDEN NO ES CAPRICHOSO

El CID de la imagen va *dentro* del JSON, así que el JSON se escribe después. Si mañana cambian la imagen, cambia su CID; entonces hay que cambiar el JSON, que tendrá otro CID, y el token seguiría apuntando al JSON viejo. Eso es el direccionamiento por contenido: nada cambia a escondidas.

5.5 · (Para la reflexión) Calculen un CID a mano. El script calcula el CID «raw» de un archivo sin subirlo a ningún lado:

```
node scripts/s11/cid-raw.js --texto "hello world"
CID raw : bafkreifzjut3te2nhyekklss27nh3k72ysco7y32koao5eei66wof36n5e
node scripts/s11/cid-raw.js ruta\...\diploma.json
```

El primero lo pueden abrir en <https://ipfs.io/ipfs/bafkreifz...n5e> : devuelve «hello world» (alguien en el mundo fijó esos mismos bytes). Comparen el CID que da el script para su `diploma.json` con el que les dio Pinata. La pregunta 3 de la sección 13 es sobre eso.

08 Paso 6 · Ensayo en la red local

10 MIN · NO GASTA NADA

Antes de gastar ETH de prueba, ensayen el despliegue completo en la red simulada de Hardhat. El módulo de Ignition despliega `DiplomaUSB` con un tope y además llama `emitir` una vez:

```
npx hardhat ignition deploy ignition/modules/s11-DiplomaUSB.js
```

Salida esperada (la dirección es siempre esta en una red local nueva):

```
Batch #1
  Executed DiplomaModulo#DiplomaUSB
Batch #2
  Executed DiplomaModulo#emitirPrimerDiploma

[ DiplomaModulo ] successfully deployed 🚀

Deployed Addresses
DiplomaModulo#DiplomaUSB - 0x5FbDB2315678afecb367f032d93F642f64180aa3
```

Cómo sé que funcionó: aparecen las dos ejecuciones (*DiplomaUSB* y *emitirPrimerDiploma*) y la dirección. Si falla aquí, fallará igual en Sepolia: arréglenlo antes de seguir.

QUÉ HIZO IGNITION

Leyó tres parámetros con sus valores por defecto: `tope = 100`, `destinatario` = la primera cuenta de Hardhat, y un `tokenURI` de ejemplo. En Sepolia los van a reemplazar por los suyos con un archivo de parámetros.

09 Paso 7 · Desplegar y verificar en Sepolia

15 MIN

7.1 · Creen su archivo de parámetros en `ignition/parametros/diploma-sepolia.json`. Solo lleva datos públicos: el tope, SU dirección (la de MetaMask, que recibirá el diploma) y SU tokenURI.

```
{
  "DiplomaModulo": {
    "tope": 100,
    "destinatario": "0xSU_DIRECCION_DE_METAMASK",
    "tokenURI": "ipfs://SU_CID_DEL_JSON"
  }
}
```

La clave privada **NO** va aquí: Hardhat la toma del keystore.

7.2 · Desplieguen, emitan y verifiquen con un solo comando (en PowerShell, todo en una línea; en bash pueden partirlo con `\`):

```
npx hardhat ignition deploy ignition/modules/s11-DiplomaUSB.js --network sepolia --parameters
ignition/parametros/diploma-sepolia.json --verify
```

Hardhat les pedirá la contraseña del keystore y confirmar que quieren desplegar en `sepolia`. Cada lote espera a que Sepolia mine (unos 12 s por bloque): **tarda uno o dos minutos**. Salida esperada:

```
[ DiplomaModulo ] successfully deployed 🚀
Deployed Addresses
DiplomaModulo#DiplomaUSB - 0x...SU_CONTRATO...
Verifying contract "contracts/s11/DiplomaUSB.sol:DiplomaUSB" for network sepolia...
Contract verified successfully ...
```

Anoten la dirección. Ignition guarda el registro en `ignition/deployments/chain-11155111/`: si la conexión se corta, repitan el mismo comando y retoma donde quedó, sin desplegar dos veces.

PUNTO DE CONTROL 3 · EN EL EXPLORADOR

Abran `https://sepolia.etherscan.io/address/0xSU_CONTRATO`. Deben ver dos transacciones (creación y `emitir`) y la pestaña *Contract* con marca de verificado. En *Read Contract*, llamen `tokenURI` con `1`: debe devolver su `ipfs://...`. Copien el hash de la transacción de `emitir`: es evidencia.

8.1 · En los exploradores. Cambien la dirección por la de su contrato:

URL	QUÉ DEBEN VER
<code>sepolia.etherscan.io/nft/0xSU_CONTRATO/1</code>	El token #1 como ERC-721, su dueño (ustedes) y la metadata si Etherscan ya la resolvió.
<code>eth-sepolia.blockscout.com/token/0xSU_CONTRATO/instance/1</code>	Explorador abierto: resuelve el JSON y muestra la imagen y los atributos.

Los exploradores resuelven la metadata en segundo plano: si la imagen no aparece enseguida, esperen unos minutos. Si al rato sigue sin aparecer, el paso 8.2 les dice cuál salto falla.

8.2 · Con el script, a la vista. En PowerShell:

```
$env:DIPLOMA="0xSU_CONTRATO"; $env:TOKEN_ID="1"
npx hardhat run scripts/s11/leer-diploma.js --network sepolia
```

(macOS/Linux: `DIPLOMA=0x... TOKEN_ID=1 npx hardhat run scripts/s11/leer-diploma.js --network sepolia`.) Salida esperada, con sus valores:

```
1 · EN LA CADENA (sepolia)
contrato  : 0x... «Diploma USB»
token     : #1
dueño     : 0x...SU_DIRECCION
tokenURI  : ipfs://SU_CID_DEL_JSON

2 · LA METADATA (el JSON)
resuelto en: https://gateway.pinata.cloud/ipfs/SU_CID_DEL_JSON
name       : Diploma de prueba · Laboratorio 11
description: Credencial de PRUEBA emitida en clase. No contiene datos personales.
image      : ipfs://SU_CID_DE_LA_IMAGEN
rasgo      : Programa = Ingeniería de Sistemas
...

3 · EL ARCHIVO (la imagen)
resuelto en: https://gateway.pinata.cloud/ipfs/SU_CID_DE_LA_IMAGEN
tamaño     : (los bytes de su PNG)
sha256     : 0x...

Los tres saltos resolvieron. Guarden esta salida como evidencia.
```

Si una puerta responde 429 o falla, el script lo dice (· `https://ipfs.io/ipfs` respondió 429; pruebo la siguiente) y sigue con otra. Si tienen puerta propia de Pinata, pueden ponerla primero con `$env:GATEWAY="https://SU-SUBDOMINIO.mypinata.cloud/ipfs"` .

CÓMO SÉ QUE TERMINÉ EL LABORATORIO

Tengo: (1) captura de `11 passing`; (2) los dos CID abriendo en una puerta; (3) contrato verificado en Sepolia y hash de `emitir`; (4) la salida completa de `leer-diploma.js` terminando en «Los tres saltos resolvieron» y una captura del token en Etherscan o Blockscout; (5) las respuestas de la sección 13.

11 Reto opcional · El diploma como SBT (ERC-5192)

EN CASA

Un ERC-721 normal se puede vender o regalar. Un diploma, no debería. ERC-5192 (estado *Final*) define tokens «atados al alma»: añade `locked(tokenId)` y los eventos `Locked / Unlocked`, y exige que, si `locked` devuelve `true`, toda función de ERC-721 que mueva el token entre cuentas revierta. Su identificador ERC-165 es `0xb45a3c0e`.

Requisito: `DiplomaUSB` completo (el SBT hereda de él). Luego:

```
copy andamiaje\s11\DiplomaSBT.sol contracts\s11\DiplomaSBT.sol
npx hardhat test test/s11/DiplomaSBT.test.js # al empezar: 2 pasan, 7 fallan
```

TODO	QUÉ HACER
1 · locked	<code>_requireOwned(tokenId);</code> (revierte si no existe) y <code>return true;</code>
2 · revocar	<code>_burn(tokenId);</code> y <code>emit DiplomaRevocado(tokenId, motivo);</code>
3 · _update (antes)	<code>address from = _ownerOf(tokenId);</code> — si <code>from</code> y <code>to</code> son distintos de cero, es una transferencia: <code>revert DiplomaIntransferible(tokenId);</code>
4 · _update (después)	Guardar <code>super._update(...)</code> en una variable; si <code>from == address(0)</code> (fue una emisión), <code>emit Locked(tokenId);</code> y devolver la variable.

Terminado: `9 passing`. Miren con atención la prueba «billetera perdida»: la universidad revoca el token 1 y emite el 2 a una cuenta nueva. Es la pregunta 5 de la reflexión.

12 Errores frecuentes

CASI TODO SE RESUELVE AQUÍ

SÍNTOMA	CAUSA Y SOLUCIÓN
Ninguna puerta encuentra el CID	Lo subieron como privado en Pinata. Súbanlo como público. Si es público: revisen que copiaron el CID completo, sin espacios.
La puerta responde 429	Límite de consultas de la puerta pública. Prueben otra (<code>gateway.pinata.cloud</code> , <code>dweb.link</code>) o la puerta propia de su cuenta.
El JSON abre pero la imagen no	"image" está mal: debe ser <code>ipfs://CID</code> (sin <code>/ipfs/</code> de más ni URL de puerta) y con el CID de la imagen, no el del JSON.
<code>SyntaxError</code> al validar el JSON	Coma de más al final de una lista, o comillas tipográficas (" ") en vez de rectas (" "). Escribanlo en VS Code, no en Word.
Pruebas: <code>ERC721NonexistentToken(1)</code>	Falta <code>_safeMint</code> (TODO 3): el token nunca se crea.
Pruebas: «Expected event DiplomaEmitido»	Falta el <code>emit</code> o los argumentos van en otro orden: <code>(tokenId, a, uri)</code> .
<code>TopeAlcanzado</code> al desplegar o emitir	El <code>tope</code> del JSON de parámetros es 0 o menor que los diplomas emitidos.
<code>OwnableUnauthorizedAccount</code>	Están emitiendo desde una cuenta distinta de la que desplegó el contrato.
HHE... «configuration variable ... not found»	Falta cargar una clave en el keystore: <code>npx hardhat keystore set NOMBRE</code> .
<code>insufficient funds for gas</code>	La cuenta de <code>SEPOLIA_PRIVATE_KEY</code> no tiene ETH de Sepolia. Pidan a la reserva del docente.
La verificación falla pero el despliegue no	Clave de Etherscan ausente o inválida. Repitan el mismo comando de Ignition con <code>--verify</code> : no redespiega.
El explorador muestra el token sin imagen	Aún no resolvió la metadata (esperen), o un salto falla: <code>leer-diploma.js</code> dice cuál.
Falta la variable <code>DIPLOMA</code>	En PowerShell es <code>\$env:DIPLOMA="0x..."</code> en la misma terminal donde corren el script.

13 Preguntas de reflexión

SE RESPONDEN MIRANDO SU PROPIO ARTEFACTO

Cada respuesta debe citar un dato de **su** despliegue (una dirección, un CID, un hash, una línea de su salida). Una respuesta genérica, sin ese dato, no puntúa.

1. **Los tres saltos.** Copien el tokenURI que devuelve su contrato y el `"image"` de su JSON. ¿Cuántos CID distintos hay en su diploma, y cuál de ellos quedó escrito en la cadena?
2. **Permanencia.** Si mañana borran el archivo de su cuenta de Pinata (dejan de fijarlo), ¿qué cambia en `ownerOf(1)` y en `tokenURI(1)` de su contrato? ¿Qué dejaría de funcionar en la salida de `leer-diploma.js`, y en qué salto?
3. **El CID no es único por archivo.** Corran `cid-raw.js` sobre su `diploma.json`. ¿Coincide con el CID que les dio Pinata? Si no, expliquen por qué ambos pueden ser «el hash del contenido» a la vez.
4. **Transferir.** Busquen en su contrato (en el explorador o en el código) la función con la que un dueño podría pasar su diploma a otra persona. ¿Qué cambiaría en la cadena y qué no cambiaría en IPFS?
5. **SBT.** Con la prueba «billetera perdida» del reto (o leyéndola en `test/s11/DiplomaSBT.test.js`): después de revocar y reemitir, ¿qué id tiene el diploma nuevo y qué pasa con el 1? ¿Quién tiene el poder real sobre un diploma SBT?
6. **Habeas data.** Miren su JSON y su transacción de `emitir` en el explorador. Señalen cada dato que es público. ¿Alguno permitiría identificar a una persona? ¿Qué NO pondrían nunca ahí, y dónde lo guardarían?

14 Lista de chequeo de entrega

✓	ÍTEM
☐	Captura de <code>npx hardhat test test/s11/DiplomaUSB.test.js</code> con <code>11 passing</code> .
☐	CID de la imagen y URL de puerta que la abre.
☐	CID del JSON y URL de puerta que lo abre. El JSON no tiene ningún dato personal.
☐	Dirección del contrato <code>DiplomaUSB</code> en Sepolia, verificado (enlace al explorador).
☐	Hash de la transacción de <code>emitir</code> (enlace al explorador).
☐	Salida completa de <code>leer-diploma.js</code> que termina en «Los tres saltos resolvieron».
☐	Captura del token en Etherscan o Blockscout.
☐	Respuestas a las seis preguntas de la sección 13, cada una con un dato propio.
☐	En el repositorio: su <code>DiplomaUSB.sol</code> y su <code>diploma.json</code> . No suban <code>.env</code> , claves ni frases (revisen <code>git status</code> antes del push).
☐	(Opcional) <code>9 passing</code> del reto SBT.

15 Rúbrica

CRITERIO	PESO	EXCELENTE (5,0)	ACEPTABLE (3,5)	INSUFICIENTE (<3,0)
Pruebas	20 %	11 verdes con su propio código; el orden CEI respetado.	11 verdes, pero con <code>_safeMint</code> antes de los efectos.	Pruebas en rojo o modificadas.
IPFS	20 %	Dos CID públicos; JSON válido y sin datos personales; <code>image</code> con <code>ipfs://</code> .	Funciona, pero <code>image</code> usa la URL de una puerta.	CID privado, roto, o JSON con datos personales.
Despliegue	25 %	Verificado en Sepolia; diploma emitido con su tokenURI real.	Desplegado sin verificar, o tokenURI de ejemplo.	Sin evidencia en cadena.
Tres saltos	20 %	Script completo y token visible en un explorador.	Solo uno de los dos.	Ninguno.
Reflexión	15 %	Seis respuestas con datos propios y razonamiento correcto.	Respuestas correctas pero genéricas.	Faltan o son incorrectas.

16 Referencias

- EIP-721, *Non-Fungible Token Standard*. eips.ethereum.org/EIPS/eip-721
- EIP-1155, *Multi Token Standard*. eips.ethereum.org/EIPS/eip-1155
- ERC-5192, *Minimal Soulbound NFTs (Final)*. eips.ethereum.org/EIPS/eip-5192
- ERC-2981, *NFT Royalty Standard (Final)*. eips.ethereum.org/EIPS/eip-2981
- IPFS, *Best practices for storing NFT data*. docs.ipfs.tech/how-to/best-practices-for-nft-data/
- Pinata, *Private IPFS y Uploading files*. docs.pinata.cloud
- OpenSea, *Farewell testnets (23-07-2025)*. support.opensea.io/en/articles/11833955-farewell-testnets
- OpenZeppelin Contracts 5.x, ERC-721. docs.openzeppelin.com/contracts/5.x/erc721
- Ley Estatutaria 1581 de 2012, régimen general de protección de datos personales (Colombia).